Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;

    // Input: Number of processes and resource types
    printf("Enter number of processes: \n");
    if (scanf("%d", &n) != 1) return 0;

    printf("Enter number of resource types: \n");
    if (scanf("%d", &m) != 1) return 0;

    int alloc[n][m], max[n][m], avail[m], need[n][m];

    // Input: Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Input: Max Matrix
```

Test Cases 2

Run Sample Cases

Run All Cases

Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```c
    printf("Enter Max Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Input: Available Resources
    printf("Enter Available Resources: \n");
    for (int j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
    }

    // Calculate Need Matrix: Need[i][j] = Max[i][j] - Allocation[i][j]
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    bool finish[n];
    for (int i = 0; i < n; i++) finish[i] = false;

    int safeSeq[n];
```

Test Cases 2

Run Sample Cases

Run All Cases

Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```
49      int count = 0;
50
51      // Banker's Safety Algorithm
52      while (count < n) {
53          bool found = false;
54
55          for (int i = 0; i < n; i++) {
56              if (!finish[i]) {
57                  bool possible = true;
58
59                  // Check if Need <= Available
60                  for (int j = 0; j < m; j++) {
61                      if (need[i][j] > avail[j]) {
62                          possible = false;
63                          break;
64                      }
65                  }
66
67                  if (possible) {
68                      // Pretend to allocate and release resources
69                      for (int j = 0; j < m; j++) {
70                          avail[j] += alloc[i][j];
71                      }
72
73                      safeSeq[count++] = i;
```

Test Cases 2

Run Sample Cases

Run All Cases

Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```
                    safeSeq[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }

        // If no process could be completed in this pass, we are stuck
        if (!found) break;
    }

    // Final Output
    if (count == n) {
        printf("System is in a SAFE STATE.\n");
        printf("Safe Sequence: ");
        for (int i = 0; i < n; i++) {
            printf("P%d ", safeSeq[i]);
        }
        printf("\n");
    } else {
        printf("System is NOT in a safe state (Deadlock may occur).\n");
    }

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases